

# Lavoisier: the method

The main features — a technical tour

Tony Walmsley

October 2026

# About this deck

- Companion to the overview deck: *how* the method works, feature by feature
- Deliberately light on code — the depth lives elsewhere:
  - docs/tutorials/ 00–06 (including a Rust on-ramp) and the learn/ exercises
  - docs/modellers-guide.md — the practical handbook
  - case-studies/ and the model crates themselves
- Everything shown here exists and is tested in the repository

# The stack

- `instructions.md` — the 22 agreed rules (R1–R22), the normative spec
- `model-core` — the library: numbers, units, resource kernel, boundary, history, requirements
- Downstream **model crates** — per system modelled: the pilot workshop plus the six case studies (CS-1..CS-6)
- The evidence loop: *experiment*  $\rightarrow$  *finding* (*F-NNN*)  $\rightarrow$  *rule* (*R-NN*)  $\rightarrow$  *implementation*
- One repository, one verification gate (`ci.sh`)

# The core mapping

| Systems engineering concept | Type-system construct                              |
|-----------------------------|----------------------------------------------------|
| Resource                    | Sealed type (no public constructor)                |
| Work step / process         | Function: everything moved in, everything returned |
| Requirement                 | Named trait, REQ-NNN, used as a bound              |
| Characteristic              | Marker trait / type parameter with kind trait      |
| Quantity                    | Integer constant in the type (base units)          |
| Processing state            | Its own distinct type                              |
| System boundary             | Module and crate privacy                           |

# Conservation by ownership

- Resources cannot be duplicated (no Clone/Copy) and cannot be shared (no borrows in process signatures)
- A process takes its inputs **by value** and returns everything it produces — including waste and the reusable tools
- One resource is in one process at a time: the borrow checker enforces exclusivity
- The modeller's translation: *“use of moved value”* means *“this resource is already in use”*

# Silent loss: a layered defence

- Rust's one gap: a value can be *dropped* silently (affine types)
- Layer 1 — compile time: `must_use` everywhere, deny-lints, a banned-methods list
- Layer 2 — test time: a **tripwire destructor** panics if a resource dies unconsumed, naming it
- Layer 3 — branch-coverage tests (branch coverage *is* leak coverage)
- The set only works complete: each lint's official fix-it suggestion is the next leak path
- Residual, documented: losses during panic unwinding

# Quantities and units

- Integers in base units: g, mm, ms, J, mm<sup>2</sup>, mm<sup>3</sup>, mK, mA, pence
- Mixing units is a **type error** — grams cannot meet millimetres
- Splits and combines carry balance checks: inputs must equal outputs, checked by the compiler
- The compiler cannot compute totals on stable Rust: callers state them, the checker verifies them

# One type per processing state

- A plate, a drilled plate and a scrap plate are three different types
- “Expected DrilledPlate, found Plate” reads as: *these plates have not been drilled yet*
- Using a part that no earlier step produced is therefore impossible to express
- Failure states (scrap, broken tool) are states too — each with a repair or disposal route

# Characteristics, catalogues, requirements

- A catalogue is one parameterised type: `Bolt<Size, Material, Length>`
- Kind traits police each slot — transposed parameters are caught at construction
- Requirements are always expressed as **trait bounds**, never as concrete types
- Adding a catalogue value costs two or three lines; the compiler re-checks every use

# Traceability, mechanically

- Each requirement is a trait whose documentation begins with its id
- Satisfies: claims are backed by compile-checked assertions — a stale claim breaks the build
- Tests carry Verifies: tags; a plain shell script produces the full traceability report
- Any requirement without a verifying test **fails CI**

# Errors phrased for modellers

- Error messages are a designed feature, curated per trait:

*“this person may not drill: Person<5000> is not a certified drilling operator (REQ-004)”*

*“BoltBox<Nil> cannot supply anything: it is empty”*

- A shipped error-reading guide translates the rest (e.g. borrow-checker vocabulary)
- Rule of the house: **ignore the compiler's fix-it suggestions** — each one is a conservation violation

# The system boundary

- Resources enter only through **suppliers**, leave only through **consumers**
- Unknowns are **placeholders**: named, greppable assumptions — “*Placeholder: atmosphere — assumed unbounded sink for exhaust and heat*”
- Refining a placeholder into the real supplier is the designed evolution path
- Unbounded sources/sinks are legal *only* at the boundary — inside the model nothing is infinite

# Discrete supply: capacity in the type

- A box of 100 bolts is a type one hundred levels deep — **on purpose**
- Taking a bolt changes the box's type ( $100 \rightarrow 99$ ); an empty box cannot supply
- The empty box is itself a resource that must be disposed of — accounted to the end
- Practical limits measured: comfortable to  $\sim 500$  items per container

# Continuous resources

- Gas, water, energy, money: a sealed container drawn down by explicit draw steps
- Overdrawing has **no valid remainder** — it cannot compile
- States can carry several quantities at once (a boiling kettle: mass *and* embodied energy)
- The same machinery serves a gas bottle, a battery, a bank account

- A person carries a time budget in milliseconds; work steps draw from it explicitly
- Overspending a budget fails exactly like overdrawing a gas bottle
- Wall-clock time and scheduling stay deliberately out of scope: the model fixes *connections*, not calendars

# History: the past as a sink

- Expended time must go somewhere: it is consumed by **History**
- History keeps an attributed record — which process drew what — forming an execution record
- Concurrent branches carry separate histories, merged at joins
- The merged record is honestly a **partial order**: no false interleaving is invented

- A step that can fail returns *either* outcome — and **both arms balance the books**
- Scrap and broken tools are outputs, not disappearances; each failure state has an exit
- Variability enters only at the boundary, as sealed outcome tokens — flows cannot peek
- Panicking past the failure arm (`unwrap`) does not compile
- Rework is bounded by provisioned reserves: unbounded retry is inexpressible

- A qualified person is a certified wrapper around the person — certification conserves them
- A fitted machine guard is a different type from an unfitted one; the process demands the fitted one
- Each currency is its own dimension; cross-currency arithmetic is a type error
- Vendors accept **only their exact price** — wrong payments are caught in the editor
- Currency exchange is integer-exact: silent rounding is unrepresentable

# Flows: connections, not sequences

- Processes declare what they consume and produce; **any order that satisfies the data wins**
- Independent branches may run concurrently — the checker proves they share no resource
- Contention is reported at the exact line: the second use of a busy resource will not compile

# The verification gate

- Eight steps, one command: full build, all tests, strict lints, a plain production build, the traceability report, a feature-placement audit, the model linter, the specification gate
- Why a *full build*: the balance checks fire late in compilation — quick checks and editors cannot see them
- Why a *plain build*: proves test fixtures cannot leak into production models
- The last two steps gate the paperwork: stale analysis reports fail; an unbalanced spec line fails **before any code exists**
- Over 230 checks, green end to end, on every change

- 15 controlled experiments; 66 findings, each citing verbatim compiler evidence
- Six case studies — pot of tea to two-site fulfilment, then a bread batch with rework-free fallibility (a scorched loaf is final) — ran the method end to end; CS-4's change-impact probes measured the “impact analysis for free” claim
- Highlights: balance checks invisible to editors (F-001); the silent-drop gap (F-002); a reproducible compiler crash found and fenced (F-034); a spec-clean scaffold that emitted silently wrong code — four generator defects found and pinned at the first field test (F-066)
- Every rule in the method cites the finding that justifies it — and the limitations are part of the record

# Where to go next

- Learn: tutorials 00–06 → `learn/` exercises → the case-study code (CS-1..CS-6) → the modeller's guide
- Generated from every model: diagrams (`docs/diagrams/`), work instructions (`docs/processes/`), lint reports (`docs/analysis/`)
- Specs are machine-checked (`speccheck`) and scaffold into model crates (`specgen`) — field-tested on CS-6, where 60 % of the scaffold survived verbatim into the finished model; the full DSL notation is validated and deliberately parked
- Read the white paper (`docs/white-paper.md`, v0.4) for the full account
- Or simply: open the repository and run `./ci.sh`